TalentTrace AI

Parent - Child Prompt Documentation

Feature-wise development prompt hierarchy and verified runtime LLM prompt boundaries

DOCUMENT TYPE	Repository-grounded prompt register
REPOSITORY	shafi-mahmud/TalentTrace-AI
SOURCE BASELINE	develop @ c4e8c90237fb5157e31d69706dc70913ffd25771
REVIEW DATE	29 October 2026
DEVELOPMENT PROMPTS	Representative reconstructions, not historical session transcripts

How to read this document

Parent prompt: a high-level feature objective that establishes scope and guardrails. Child prompt: a smaller, testable engineering instruction that implements part of that objective. The parent-child wording below has been reconstructed to explain the development workflow; it was not extracted from historical chats.

The second part describes system/user prompt boundaries actually present in the application source. A feature can use LangGraph without an LLM prompt, and a deterministic service is not automatically an agent.

SUBMISSION INTEGRITY
Do not label reconstructed prompts as exact prompts entered in Codex/ChatGPT. Obtain chat/session exports before making that claim.

A. Development prompt hierarchy

These examples are reconstructed from the verified code architecture and feature documentation. Each group follows the same broad pattern: define a feature, implement a narrow slice, test safety/failures, document evidence.

P01 Platform foundation and authentication

PARENT - reconstructed example
Design a beginner-explainable recruiting workspace with React, FastAPI, PostgreSQL and recruiter-scoped job data. Implement in small reviewed increments. Keep secrets and candidate data private.

CHILD P01.1 Define a minimal frontend/API/database layout, environment configuration and migrations before adding agents.

CHILD P01.2 Add authenticated recruiter sessions and enforce job ownership on read/write endpoints; test foreign-user denial.

CHILD P01.3 Document request flow, synthetic test fixtures, security assumptions and meaningful commits.

Code/document evidence: README.md; backend/app/api/routers/auth.py; backend/app/api/routers/jobs.py

Acceptance boundary: Foundation exists; not a claim of full security certification.

P02 Job descriptions, recruiter chat and policy RAG

PARENT - reconstructed example
Build AI-assisted job profile drafting with a reviewable conversation and grounded company-policy context. Keep recruiter approval mandatory before criteria are used.

CHILD P02.1 Create persistent recruiter-owned job identities and revision-aware JD refinement endpoints.

CHILD P02.2 Build a LangGraph chain: retrieve_policy_context -> generate_draft -> validate_draft, with schema-validated output and citation checks.

CHILD P02.3 Parse recruiter refinements from trusted server-held conversation context without treating user text as system instructions.

CHILD P02.4 Test missing provider, malformed JSON, unknown citations and ownership isolation.

Code/document evidence: backend/app/agents/jd/workflow.py; backend/app/integrations/llm/groq.py; backend/app/integrations/llm/recruiter_chat.py

Acceptance boundary: Application uses LLM prompts; outputs still require recruiter review.

P03 Job-circular publishing via MCP / adapters

PARENT - reconstructed example
Allow a recruiter to review a saved job circular and explicitly publish through authorized destinations. Never silently post to social platforms.

CHILD P03.1 Add a publication preview tied to the current saved Job Reference and revision.

CHILD P03.2 Integrate the existing Discord MCP adapter; handle receipts, failures and uncertain duplicate requests.

CHILD P03.3 Support configured Telegram dispatch and a manual-copy workflow for other destinations.

CHILD P03.4 Test authorization, wrong-job data, duplicate attempts and disabled destinations with synthetic providers.

Code/document evidence: docs/multiplatform_job_publishing.md; docs/jd_chat_publishing_20261028.md

Acceptance boundary: Direct automated LinkedIn/X publishing is not implemented.

P04 CV intake, OCR and grounded candidate parsing

PARENT - reconstructed example
Implement safe batch PDF/DOCX intake, extracting usable source text and evidence-linked candidate qualifications. Treat every CV as untrusted input.

CHILD P04.1 Validate file size, format, paths and recruiter/job ownership; store uploads privately.

CHILD P04.2 Extract digital PDF/DOCX text, add Tesseract fallback for image-heavy PDFs, and preserve page/block source references.

CHILD P04.3 Use structured LLM parsing to extract supported skills, education and experience with verbatim source evidence.

CHILD P04.4 Add tests for malformed documents, prompt-injection text, extraction failures and missing evidence.

Code/document evidence: docs/cv_ingestion.md; backend/app/integrations/llm/candidate_profile.py; backend/app/services/cv_ocr.py

Acceptance boundary: Batch upload supports 20 files; this does not imply all 20 receive numeric scores.

P05 F1 LangGraph screening and checkpoint support

PARENT - reconstructed example
Build conservative candidate screening from recruiter-approved criteria using deterministic evidence matching, optional semantic interpretation and explicit eligibility outcomes.

CHILD P05.1 Create LangGraph nodes for input validation, requirement context and candidate-source readiness.

CHILD P05.2 Match direct evidence; route unresolved criteria through a bounded semantic LLM call; validate referenced evidence.

CHILD P05.3 Route mandatory checks to eligible, excluded or insufficient_evidence; preserve no_decision for failures.

CHILD P05.4 Add a separate PostgreSQL checkpointed start/resume interface and test source/identity mismatch.

Code/document evidence: backend/app/agents/screening/workflow.py; backend/app/agents/screening/checkpointed.py; docs/checkpoint_storage.md

Acceptance boundary: Full screening-specific cross-process recovery and all-candidate numeric ranking remain partially accepted.

P06 F2 weighted scoring and ranking

PARENT - reconstructed example
Provide explainable 0-100 requirement-coverage scores from validated F1 evidence, not from an invented applicant-fit judgment.

CHILD P06.1 Use approved skills / experience / education weights that total 100; start from provisional 50/30/20 defaults.

CHILD P06.2 Return not_evaluated rather than a fabricated zero when evidence is uncertain or an applicant is excluded.

CHILD P06.3 Persist score breakdowns and job/batch/revision provenance; display ordered numeric rankings and ties.

CHILD P06.4 Test arithmetic, missing dimensions, uncertainty, repeatability and cross-job isolation.

Code/document evidence: backend/app/services/candidate_scoring.py; docs/f2_scoring.md; docs/f1_f2_deadline_acceptance.md

Acceptance boundary: There is no standalone ATS score or unified F2+F5 numeric score.

P07 F3 scheduling and candidate-specific access

PARENT - reconstructed example
Implement protected future interview booking from saved screening outcomes, with explicit recruiter confirmation and a candidate-specific access boundary.

CHILD P07.1 Build a timezone-aware booking UI and validate date/time/duration on the server.

CHILD P07.2 Link booking to the exact recruiter, job, batch, application and approved criteria revision.

CHILD P07.3 Issue bounded, single-use candidate invitations; enforce expiry, scope and revocation.

CHILD P07.4 Test duplicate bookings, conflict cases, stale criteria, unauthorized access and reload persistence.

Code/document evidence: docs/interview_scheduling.md; docs/interview_invitations.md; backend/app/api/routers/interviews.py

Acceptance boundary: External calendar sync and complete media-token revocation are not fully established.

P08 F3 WebRTC interview and media controls

PARENT - reconstructed example
Integrate LiveKit for authorized two-person browser audio/video interviews, while keeping microphone, camera and leave controls safe and understandable.

CHILD P08.1 Generate short-lived interview-scoped role tokens; connect recruiter and candidate using separate sessions.

CHILD P08.2 Publish/subscribe to local and remote audio/video; provide mute, camera toggle and Leave.

CHILD P08.3 Clean up media resources on disconnect and deny unauthorized rejoining.

CHILD P08.4 Test with synthetic generated media first, then separately record real transport acceptance and remaining reconnect limits.

Code/document evidence: frontend/src/components/LiveInterviewPage.tsx; docs/task3_acceptance_20261027.md

Acceptance boundary: Cloud two-party media was verified; automatic network recovery and self-hosted reliability remain partial.

P09 F4 speech-to-text and transcript privacy

PARENT - reconstructed example
Add independent, consent-gated recruiter/candidate caption streams; process temporary audio in bounded memory and optionally persist finalized text under separate storage consent.

CHILD P09.1 Capture mono PCM using AudioWorklet and authenticated role-specific WebSockets.

CHILD P09.2 Support local Faster-Whisper and separately configured AssemblyAI with explicit provider/consent boundaries.

CHILD P09.3 Store validated role/time/source transcript turns only after both participants consent; support withdrawal and deletion.

CHILD P09.4 Test slow providers, audio overflow, stale turns, duplicate events, role isolation and retention.

Code/document evidence: docs/f4_free_captions.md; docs/f4_transcript_integration.md; docs/task4c_transcript_persistence_20261027.md

Acceptance boundary: Real API recognition and offline browser persistence passed separately; full live combined path remains partial.

P10 F5 grounded post-interview evaluation

PARENT - reconstructed example
Build a LangGraph analyst that pairs trusted interview questions/answers, maps them to job criteria, retrieves curated technical references and produces advisory, source-grounded assessments.

CHILD P10.1 Validate complete consented transcript input; segment Q&A with conservative unresolved/uncertain cases.

CHILD P10.2 Map questions to approved requirements and retrieve bounded reference documents.

CHILD P10.3 Assess relevance, technical accuracy, completeness and articulation; validate every cited transcript quote and technical reference.

CHILD P10.4 Allow one bounded grounding repair, reject unsupported output and persist results with input fingerprints.

Code/document evidence: backend/app/agents/evaluation/workflow.py; backend/app/integrations/llm/interview_evaluation.py; docs/milestone3_interview_analysis.md

Acceptance boundary: Current full live F4 -> F5 acceptance remains partial; no autonomous hiring decisions.

P11 F6 consolidated assessment and comparison

PARENT - reconstructed example
Present recruiter-authorized CV and interview evidence together without inventing a combined suitability number or disclosing private transcripts to unauthorized users.

CHILD P11.1 Retrieve F2 records and authorized F5 analysis for the exact application and criteria revision.

CHILD P11.2 Show strengths, concerns, missing evidence and source links with clear uncertainty labels.

CHILD P11.3 Add same-job two-candidate comparison and browser Print/Save as PDF with private cleanup.

CHILD P11.4 Test foreign-owner rejection, withdrawn transcripts, print lifecycle and missing F5 evidence.

Code/document evidence: docs/f6_consolidated_assessment.md; backend/app/services/consolidated_assessment.py; frontend/src/api/assessmentPrinting.ts

Acceptance boundary: Two-candidate comparison exists; no approved unified 0-100 ranking across F2/F5.

P12 F11 live interviewer co-pilot

PARENT - reconstructed example
Provide a recruiter-only, optional follow-up suggestion assistant using the trusted F4 caption stream and confirmed job requirements. Abstain on ambiguity and never infer candidate competence.

CHILD P12.1 Implement a model-free LangGraph: validate_context -> assemble_dialogue -> track_exploration -> select_follow_up -> validate_output.

CHILD P12.2 Bind each run to interview/caption IDs and criteria revision; require separate recruiter and candidate consent.

CHILD P12.3 Render provisional suggestions only to the recruiter; provide dismiss, expiry, withdrawal and stale-event protection.

CHILD P12.4 Verify candidate exclusion and UI lifecycle using synthetic captions; disclose unverified live-provider latency.

Code/document evidence: backend/app/agents/copilot/workflow.py; docs/f11_final_acceptance_20261027.md; docs/f11_copilot_live.md

Acceptance boundary: F11 generates template-based prompts without a runtime LLM call; real cloud F4 -> F11 remains partial.

P13 Google intake, application counts and notifications

PARENT - reconstructed example
Support narrowly authorized import of external candidate applications into the existing recruiter-owned CV intake ledger without counting applications twice or scanning unrelated records.

CHILD P13.1 Validate one selected Google Form response, consent, Job Reference and attached file metadata/bytes.

CHILD P13.2 Reuse existing private CV storage, imported-submission identity and duplicate-prevention checks.

CHILD P13.3 Show recruiter-scoped received/new/viewed application counts with explicit acknowledgement.

CHILD P13.4 Test invalid response paging, mismatched job/source, repeated imports and cross-job notification isolation.

Code/document evidence: docs/google_single_import_20261028.md; backend/app/api/routers/application_notifications.py

Acceptance boundary: One authorized synthetic Google response was imported; unattended sync is not implemented.

P14 N19 guarded candidate-submission deletion

PARENT - reconstructed example
Design a privacy-preserving administrative deletion workflow that fails closed when candidate identity or related records cannot be safely determined.

CHILD P14.1 Inventory dependencies for one exact recruiter/job/batch/upload tuple and verify a single owned CV file.

CHILD P14.2 Require reviewed plan digest, explicit confirmation and controlled maintenance privileges.

CHILD P14.3 Delete supported imported-submission records and exact file, record a minimal replay fence and check recovery paths.

CHILD P14.4 Reject linked interview/shared artifacts; test rollback, repeat operation, unrelated-data preservation and re-import suppression.

Code/document evidence: docs/n19_guarded_submission_deletion.md; docker/n19_delete.py

Acceptance boundary: This is not candidate-wide erasure or an ordinary DELETE /candidates/ API.

P15 N21 Docker release, documentation and verification

PARENT - reconstructed example
Produce a reproducible reviewer setup and an evidence-based final submission without claiming that tests not run have passed.

CHILD P15.1 Package frontend, backend and PostgreSQL in Docker Compose with private env files, explicit migrations and health checks.

CHILD P15.2 Run focused unit/integration tests, synthetic browser scenarios and current-host Docker persistence checks.

CHILD P15.3 Document configuration, architecture, security/PII cautions, acceptance boundaries and reviewer steps.

CHILD P15.4 Prepare a tagged release checkpoint, while clearly separating full BID acceptance from bounded component demonstrations.

Code/document evidence: README.md; docs/task8_release_audit_20261028.md; docs/deadline_closure_20261028.md

Acceptance boundary: Same-host fresh setup passed; second clean machine and full configured-provider deployment remain unverified.

B. Application runtime prompts

Unlike the reconstructed development prompts above, the following prompt arrangements were checked in the source. They are simplified descriptions of the runtime prompt boundaries, not full dumps of sensitive prompt payloads.

FEATURE	SYSTEM / USER INPUT	RESPONSE AND SOURCE
JD generation	System instruction + recruiter request and retrieved policy chunks	Structured JD draft with policy citation IDs integrations/llm/groq.py
Recruiter-chat interpretation	System instruction + current/prior recruiter conversation data	Validated intent and evidence spans integrations/llm/recruiter_chat.py
CV profile extraction	System instruction + untrusted source units	Source-backed structured candidate attributes integrations/llm/candidate_profile.py
F1 semantic matching	System instruction + untrusted criteria/fact JSON	One structured result per criterion and evidence references integrations/llm/screening.py
F5 assessment	System instruction + trusted Q&A mappings, transcript turns and references	Four rubric dimensions with validated evidence; bounded repair integrations/llm/interview_evaluation.py
F2, F11 and F6	No additional LLM prompt used in their core decision/rendering logic	Deterministic scoring, model-free follow-up and evidence projection services/candidate_scoring.py; agents/copilot/workflow.py; services/consolidated_assessment.py

Important distinction for viva

F11 uses LangGraph for deterministic, model-free question suggestion logic. F2 scoring and F6 consolidation are also deterministic services. Their business instructions should not be presented as actual runtime LLM system prompts.

C. Evidence and submission rules

What is verified: source files, runtime prompt boundaries, implemented feature structure, and repository-recorded test limitations at the named Git commit.

What is not verified: exact historical development prompts, which teammate entered which prompt, original conversation chronology, or full assistant responses. Git commit messages do not contain that evidence by default.

Recommended wording in the trainer report: "The following parent-child development prompts are representative reconstructions aligned with the implementation. Original prompts can be attached from the development chat history where available."

D. How to extract the exact prompts with Codex

If the trainer explicitly asks for the original prompt text, use Codex or the original chat export on the development machine. Ask for a traceable prompt list and verify each prompt manually.

> Find accessible TalentTrace AI development-session prompt history, if available. For each feature, extract verbatim user parent prompts and chronological child prompts with timestamp, session locator and related commit. Do not guess prompts from source code. If history is unavailable, write 'not recoverable'. Exclude API keys, private credentials, real CVs, candidate PII, recordings and transcripts. Export a feature-to-prompt evidence table for manual review.

Repository evidence index

- README.md
- docs/final_submission_audit_20261028.md
- docs/deadline_closure_20261028.md
- backend/app/agents/jd/workflow.py
- backend/app/agents/screening/workflow.py
- backend/app/agents/evaluation/workflow.py
- backend/app/agents/copilot/workflow.py

Prepared for the TalentTrace AI BJIT final project. This file is a documentation artifact only; it does not claim that new code or tests were executed or that the repository was changed.